Compte rendu du TP 1
Apache Spark Core et RDD

Anwar Ezzirani • EMSI • Groupe G4 • Module Big Data

Ce TP utilise Python et PySpark dans JupyterLab, avec Spark 4.0.0 exécuté dans Docker. Il porte sur la création des RDD, leurs partitions, les transformations, les actions et un comptage de mots. Les sorties ci-dessous proviennent du notebook exécuté ; les comportements non conservés dans ses sorties sont identifiés explicitement.

Partie 0 - Session Spark et environnement

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .master("local[4]")
    .appName("TP-RDD")
    .getOrCreate()
)

sc = spark.sparkContext
sc.setLogLevel("WARN")

DATA = "/home/jovyan/work"
```

Q0.1 - Configuration du SparkContext

```
print("Mode d'exécution :", sc.master)
print("Parallélisme par défaut :", sc.defaultParallelism)
print("Version :", sc.version)
```

Résultat obtenu

```
Mode d'exécution : local[4]
Parallélisme par défaut : 4
Version : 4.0.0
```

SparkSession est le point d’entrée principal de l’application. Son SparkContext, nommé sc, permet de créer les RDD. local[4] signifie une exécution locale avec quatre threads ; le parallélisme par défaut vaut 4 et la version est 4.0.0. Le dossier DATA est adapté au montage Docker utilisé : /home/jovyan/work.

Q0.2 - Interface et driver

```
print(sc.uiWebUrl)
```

Résultat obtenu

```
http://ee2194785683:4041
```

L’interface de cette session est accessible depuis Windows à http://localhost:4041. La capture de l’onglet Executors montre une seule entrée active, driver, avec 4 cores et aucune tâche exécutée au moment de la capture. En mode local, le même processus JVM héberge la coordination et l’exécution des tâches. Jobs recense les jobs, Stages leurs étapes et Executors les ressources d’exécution.

Jeu de données - Six citations

```
citations = [
    "Spark est un moteur de calcul distribue",
    "Spark traite les donnees en memoire",
    "Hadoop stocke les donnees sur disque",
    "Spark est rapide et Spark est facile",
    "Les RDD sont immuables et distribues",
    "Le calcul distribue repose sur les RDD",
]

with open(DATA + "/citations.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(citations) + "\n")

print("Fichier citations.txt créé.")
```

Résultat obtenu

```
Fichier citations.txt créé.
```

La liste contient six phrases. Python les écrit dans citations.txt, une phrase par ligne. Ce fichier se trouve dans le dossier partagé work ; cette écriture Python ne déclenche pas de job Spark.

Exercice 1.1 - Création et partitions

```
x = sc.parallelize(range(1, 21), 4)

print("Nombre de partitions :", x.getNumPartitions())
print("Contenu des partitions :", x.glom().collect())
```

Résultat obtenu

```
Nombre de partitions : 4
Contenu des partitions : [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15], [16, 17,
    18, 19, 20]]
```

parallelize crée un RDD à partir d’une collection Python. Les 20 nombres sont répartis ici en quatre partitions de cinq éléments. getNumPartitions fournit leur nombre ; glom regroupe chaque partition dans une liste et collect ramène ces listes au driver.

```
x_auto = sc.parallelize(range(1, 21))

print("Partitions automatiques :", x_auto.getNumPartitions())
print("Parallélisme par défaut :", sc.defaultParallelism)
```

Résultat obtenu

```
Partitions automatiques : 4
Parallélisme par défaut : 4
```

Sans second argument, parallelize utilise sc.defaultParallelism. Le nombre automatique de partitions est donc 4 dans cette session. Une partition est une portion des données, pas une machine ni un executor.

Exercice 1.2 - Lecture du fichier texte

```
txt = sc.textFile(DATA + "/citations.txt", 2)

print("Nombre de lignes :", txt.count())
print("Première ligne :", txt.first())
print("Type :", type(txt))
```

Résultat obtenu

```
Nombre de lignes : 6
Première ligne : Spark est un moteur de calcul distribue
Type : <class 'pyspark.core.rdd.RDD'>
```

Le RDD txt contient six éléments de type chaîne, un par ligne du fichier. first renvoie la première phrase. parallelize utilise une collection Python comme source, tandis que textFile utilise un fichier. Le second argument de textFile indique le nombre minimal de partitions demandé ; il ne constitue pas une garantie universelle de nombre exact.

Exercice 1.3 - Évaluation paresseuse

```
inexistant = sc.textFile(DATA + "/fichier_qui_n_existe_pas.txt")
print("RDD défini")
```

Résultat obtenu

```
RDD défini

inexistant.count()
```

Avant la création du fichier, cette action a produit l’erreur suivante, observée pendant la séance :

```
Input path does not exist:
file:/home/jovyan/work/fichier_qui_n_existe_pas.txt
```

La définition du RDD avait réussi ; l’erreur est apparue lors de count, quand Spark a dû accéder à la source. Cela illustre l’évaluation paresseuse : définir un RDD décrit le traitement, une action demande son exécution. Certaines opérations de consultation des métadonnées peuvent également accéder à la source.

Vérification après création du fichier

```
with open(DATA + "/fichier_qui_n_existe_pas.txt", "w", encoding="utf-8") as f:
    f.write("Bonjour Spark\n")

inexistant.count()
```

Résultat obtenu

```
1
```

Le notebook fourni conserve cette seconde exécution réussie : après création d’un fichier contenant Bonjour Spark sur une ligne, count renvoie 1. L’erreur initiale n’est plus présente dans les sorties enregistrées du notebook, mais elle a été observée avant cette correction.

Partie 2 - Transformations de base

Exercice 2.1 - Carré des éléments

```
nombres = sc.parallelize([7, 10, 12, 17, 19], 2)

carres = nombres.map(lambda a: a ** 2)

print("Carrés :", carres.collect())
print("Nombre initial :", nombres.count())
print("Nombre après map :", carres.count())
```

Résultat obtenu

```
Carrés : [49, 100, 144, 289, 361]
Nombre initial : 5
Nombre après map : 5
```

map applique a ** 2 à chaque nombre. Les carrés sont 49, 100, 144, 289 et 361. Le nombre d'éléments reste 5 : map produit un élément par entrée. Le RDD nombres n'est pas modifié.

Exercice 2.2 - Quotient et reste

```
couples = nombres.map(lambda a: (a // 2, a % 2))

print(couples.collect())
```

Résultat obtenu

```
[(3, 1), (5, 0), (6, 0), (8, 1), (9, 1)]
```

Chaque nombre devient le couple (quotient entier, reste) de sa division par 2. Par exemple, $7 = 2 \times 3 + 1$ donne (3, 1). Le reste vaut 0 pour les nombres pairs et 1 pour les impairs.

Exercice 2.3 - filter et map

```
pairs = nombres.filter(lambda a: a % 2 == 0)
tests = nombres.map(lambda a: a % 2 == 0)

print("Avec filter :", pairs.collect())
print("Avec map :", tests.collect())
```

Résultat obtenu

```
Avec filter : [10, 12]
Avec map : [False, True, True, False, False]
```

filter conserve les valeurs qui vérifient la condition : 10 et 12. Avec la même condition, map produit un booléen pour chaque nombre et conserve cinq éléments. Tester une condition avec map ne supprime donc aucun élément.

Exercice 2.4 - Déplier avec flatMap

```
groupes = sc.parallelize([(1, 2), (3, 5), (8, 10)])

plat = groupes.flatMap(lambda couple: couple)
avec_map = groupes.map(lambda couple: couple)

print("Avec flatMap :", plat.collect())
print("Avec map :", avec_map.collect())
```

Résultat obtenu

```
Avec flatMap : [1, 2, 3, 5, 8, 10]
Avec map : [(1, 2), (3, 5), (8, 10)]
```

La fonction renvoie chaque couple reçu. map le conserve comme un seul élément ; flatMap déplie ses deux composantes. Le RDD résultat contient donc six nombres au lieu de trois couples.

Exercice 2.5 - Filtrer avec flatMap

```
pairs_flatmap = nombres.flatMap(
    lambda a: [a] if a % 2 == 0 else []
)

print(pairs_flatmap.collect())
```

Résultat obtenu

```
[10, 12]
```

Pour un nombre pair, la fonction renvoie [a] ; pour un impair, elle renvoie []. flatMap déplie ces listes et les listes vides ne produisent aucun élément. Le résultat [10, 12] est identique à celui du filtre sur les nombres pairs.

Exercice 2.6 - Valeurs distinctes

```
z = sc.parallelize([1, 1, 2, 3, 4, 5, 5, 5, 6, 6])

uniques = z.distinct()

print(uniques.collect())
```

Résultat obtenu

```
[4, 1, 5, 2, 6, 3]
```

distinct supprime les doublons et produit les six valeurs uniques. L’ordre observé [4, 1, 5, 2, 6, 3] diffère de l’ordre initial : distinct ne garantit pas le maintien de cet ordre. Le RDD original conserve ses doublons.

Ces méthodes retournent de nouveaux RDD. Les sorties sont obtenues grâce à collect, qui est une action ; les transformations elles-mêmes sont paresseuses.

Partie 3 - Actions

Exercice 3.1 - Inspecter un RDD

```
y = sc.parallelize([6, 8, 9, 1, 2, 3], 3)

print("count :", y.count())
print("first :", y.first())
print("take(3) :", y.take(3))
print("take(100) :", y.take(100))
print("takeOrdered(3) :", y.takeOrdered(3))
print("collect :", y.collect())
```

Résultat obtenu

```
count : 6
first : 6
take(3) : [6, 8, 9]
take(100) : [6, 8, 9, 1, 2, 3]
takeOrdered(3) : [1, 2, 3]
collect : [6, 8, 9, 1, 2, 3]
```

count compte les éléments ; first renvoie le premier. take(3) prend les trois premiers, alors que takeOrdered(3) sélectionne les trois plus petits dans l'ordre croissant. take(100) renvoie les six éléments disponibles. collect récupère l'ensemble au driver. Cette cellule a été exécutée deux fois dans le notebook ; une seule occurrence est présentée ici.

Exercice 3.2 - Agrégations

```
print("Somme avec reduce :", y.reduce(lambda a, b: a + b))
print("Somme avec sum :", y.sum())
print("Maximum :", y.max())
print("Minimum :", y.min())
print("Moyenne :", y.mean())
```

Résultat obtenu

```
Somme avec reduce : 29
Somme avec sum : 29
Maximum : 9
Minimum : 1
Moyenne : 4.833333333333333
```

reduce combine les valeurs avec l'addition, comme sum : le total est 29. Spark combine des résultats partiels calculés dans les partitions. Le maximum est 9, le minimum 1 et la moyenne 29 / 6 = 4,8333 environ. Ces méthodes sont des actions.

Exercice 3.3 - RDD vide

```
vide = sc.parallelize([])

print(vide.take(3))
```

Résultat obtenu

```
[]
```

take(3) renvoie une liste vide car aucun élément n’est disponible. first sur un RDD vide provoque une erreur : il n’existe pas de premier élément. Ce comportement est expliqué ici ; l’appel direct vide.first() et sa sortie d’erreur ne sont pas conservés dans le notebook fourni.

Protection contre l’absence de premier élément

```
if vide.isEmpty():
    print("Le RDD est vide.")
else:
    print(vide.first())
```

Résultat obtenu

```
Le RDD est vide.
```

isEmpty vérifie si le RDD est vide. Le test empêche l’appel à first quand aucun élément n’existe et affiche Le RDD est vide.

Exercice 3.4 - Limites de collect

```
print(y.take(3))
```

Résultat obtenu

```
[6, 8, 9]
```

collect ramène tous les éléments dans la mémoire du driver. Sur un très gros RDD, cette récupération peut épuiser la mémoire. Pour consulter un petit échantillon, take(3) suffit et renvoie ici [6, 8, 9].

Partie 4 - Mini-projet Word Count

Exercice 4.1 - Découper et normaliser

```
mots = txt.flatMap(lambda ligne: ligne.lower().split(" "))

print("Nombre total de mots :", mots.count())
print("Nombre de mots distincts :", mots.distinct().count())
```

Résultat obtenu

```
Nombre total de mots : 39
Nombre de mots distincts : 25
```

lower convertit les lignes en minuscules ; split les découpe aux espaces et flatMap produit un élément par mot. Le fichier contient 39 occurrences et 25 mots distincts. Spark et spark sont ainsi comptés sous la même forme.

Exercice 4.2 - Compter les occurrences

```
comptage = (
    mots
    .map(lambda mot: (mot, 1))
    .reduceByKey(lambda a, b: a + b)
)

print(comptage.collect())
```

Résultat obtenu

```
[('est', 3), ('un', 1), ('distribue', 2), ('traite', 1), ('les', 4), ('donnees', 2),
    ('hadoop', 1), ('sur', 2), ('rapide', 1), ('et', 2), ('rdd', 2), ('sont', 1),
    ('immuables', 1), ('repose', 1), ('spark', 4), ('moteur', 1), ('de', 1), ('calcul', 2),
    ('en', 1), ('memoire', 1), ('stocke', 1), ('disque', 1), ('facile', 1), ('distribues', 1),
     ('le', 1)]
```

map crée les paires (mot, 1). reduceByKey additionne les valeurs pour chaque mot, avec combinaison locale puis shuffle. Le comptage contient une paire par mot distinct ; la somme des occurrences est 39.

Exercice 4.3 - Les cinq mots les plus fréquents

```
top5 = comptage.takeOrdered(5, key=lambda couple: -couple[1])

print(top5)
```

Résultat obtenu

```
[('les', 4), ('spark', 4), ('est', 3), ('distribue', 2), ('donnees', 2)]
```

takeOrdered utilise l’opposé du nombre d’occurrences pour sélectionner les fréquences décroissantes. Le top observé est les (4), spark (4), est (3), distribue (2), donnees (2). Plusieurs mots ont une fréquence de 2 : les deux dernières places peuvent changer en cas d’égalité.

Exercice 4.4 - Lignes et longueur maximale

```
lignes_spark = txt.filter(
    lambda ligne: "Spark" in ligne.split(" ")
)

longueurs = txt.map(lambda ligne: len(ligne.split(" ")))

print("Lignes contenant Spark :", lignes_spark.count())
print("Longueur maximale :", longueurs.max())
```

Résultat obtenu

```
Lignes contenant Spark : 3
Longueur maximale : 7
```

Le filtre cherche Spark comme mot exact, avec respect de la casse. Trois lignes contiennent ce mot, bien qu'il apparaisse quatre fois au total : une ligne le contient deux fois. La longueur maximale est de sept mots ; plusieurs lignes atteignent cette longueur.

Exercice 4.5 - Sauvegarde et relecture

```
comptage.coalesce(1).saveAsTextFile(DATA + "/wc_out")
```

coalesce(1) réduit le RDD à une partition. saveAsTextFile crée un dossier avec un fichier de données part-00000 et des fichiers techniques ; il ne crée pas un fichier unique portant le nom wc_out.

```
import os

print("Fichiers créés :", os.listdir(DATA + "/wc_out"))

resultat = sc.textFile(DATA + "/wc_out")

print("Nombre de lignes :", resultat.count())
print("Exemples :", resultat.take(5))
```

Résultat obtenu

```
Fichiers créés : ['.part-00000.crc', '._SUCCESS.crc', 'part-00000', '_SUCCESS']
Nombre de lignes : 25
Exemples : ["('est', 3)", "('un', 1)", "('distribue', 2)", "('traite', 1)", "('les', 4)"]
```

La relecture compte 25 lignes, une par mot distinct. Les lignes relues sont des chaînes représentant les couples, et non des tuples Python. _SUCCESS marque la réussite de l'écriture ; les fichiers .crc sont des fichiers de contrôle.

```
comptage.coalesce(1).saveAsTextFile(DATA + "/wc_out_2")
```

Une nouvelle sauvegarde au même emplacement wc_out échouerait parce que le dossier existe déjà. Le notebook utilise wc_out_2 pour conserver le premier résultat. L'erreur de réécriture au même chemin n'a pas été enregistrée dans les sorties fournies.

Partie 5 - Synthèse

Q5.1 - Transformations et actions

Méthode	Nature	Retour
map, filter, flatMap, distinct	Transformation	Nouveau RDD
reduceByKey	Transformation	Nouveau RDD de paires
count	Action	Nombre d'éléments
first	Action	Premier élément
take, collect, takeOrdered	Action	Liste Python
reduce	Action	Valeur agrégée
saveAsTextFile	Action	Écriture de fichiers ; None

Une transformation définit un nouveau RDD à partir des dépendances existantes. Une action demande un résultat ou une écriture, ce qui déclenche les calculs nécessaires.

Q5.2 - Jobs exécutés

Le nombre exact de jobs n'a pas été relevé dans les éléments fournis : la réponse du notebook contient encore un emplacement vide et les captures ne montrent pas l'onglet Jobs. Il doit être complété à partir de l'application TP-RDD dans Spark UI, en distinguant les jobs terminés et échoués, s'il y en a.

Les actions telles que count, collect ou saveAsTextFile peuvent déclencher un ou plusieurs jobs. Une transformation seule comme map ou filter n'en lance pas. Les réexécutions de cellules peuvent ajouter des jobs ; on ne peut donc pas déduire le total du seul nombre d'exercices.

Q5.3 - Définition d'un RDD

Un RDD (Resilient Distributed Dataset) est une collection immuable divisée en partitions que Spark peut traiter en parallèle. Spark conserve ses dépendances, appelées lineage, pour recalculer des partitions perdues lorsque les données nécessaires restent disponibles.

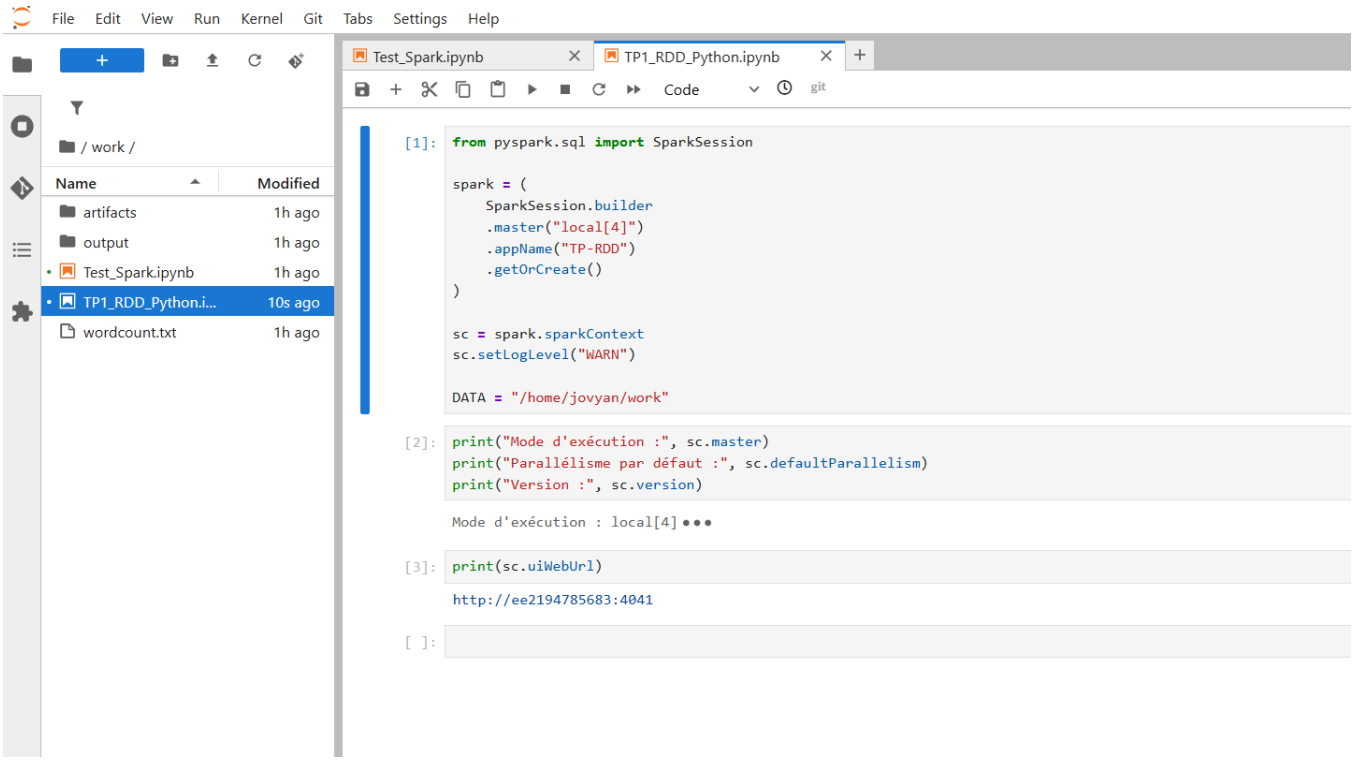
Bilan du TP

Les exercices ont permis de créer des RDD à partir de collections et d'un fichier, d'observer les partitions, puis de comparer transformations et actions. Le Word Count a produit 25 mots distincts à partir de 39 occurrences et les résultats ont été sauvegardés puis relus. L'environnement est local : aucune infrastructure HDFS ni cluster de machines n'a été utilisée.

Sources de travail : énoncé TP1_RDD_Python.docx, notebook TP1_RDD_Python.ipynb, captures de JupyterLab et de Spark UI, ainsi que l'erreur de fichier absent observée pendant la séance. Le bonus sur les mots vides n'a pas été exécuté dans le notebook fourni.

Annexe - Capture 1

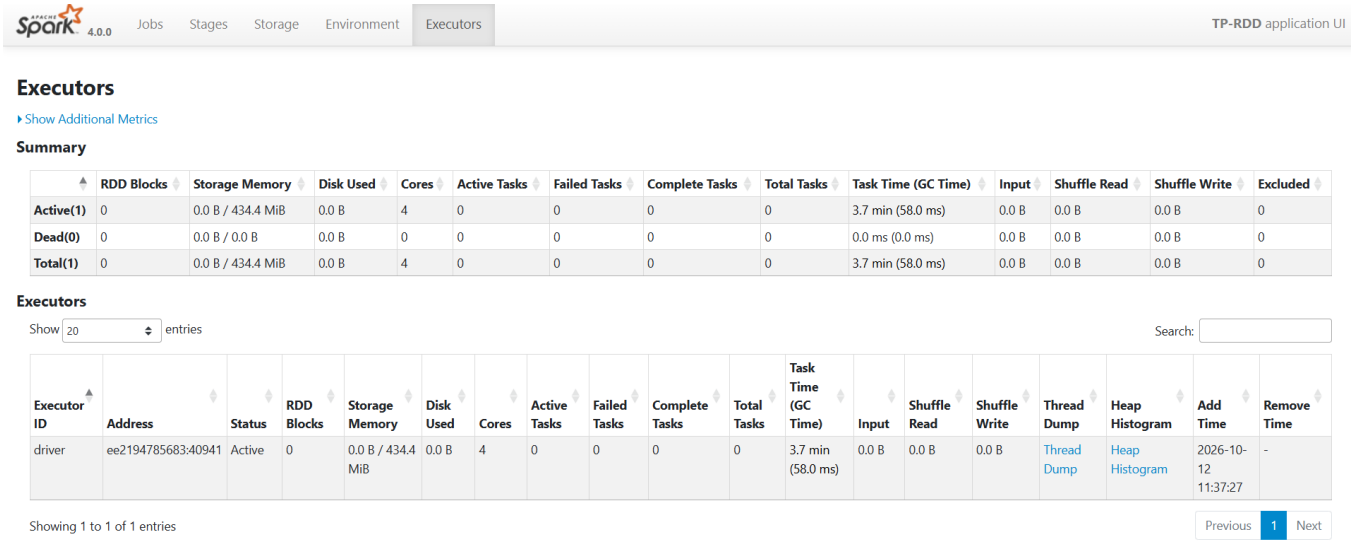
Configuration de Spark et paramètres du SparkContext



Capture fournie pendant l'exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 2

Spark UI et executor local nommé driver



Capture fournie pendant l’exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 3

Création du fichier de citations et premières partitions

[4]:
```
citations = [
    "Spark est un moteur de calcul distribue",
    "Spark traite les donnees en memoire",
    "Hadoop stocke les donnees sur disque",
    "Spark est rapide et Spark est facile",
    "Les RDD sont immuables et distribues",
    "Le calcul distribue repose sur les RDD",
]

with open(DATA + "/citations.txt", "w", encoding="utf-8") as f:
    f.write("\n".join(citations) + "\n")

print("Fichier citations.txt créé.")
```

```
Fichier citations.txt créé.
```

[6]:
```
x = sc.parallelize(range(1, 21), 4)

print("Nombre de partitions :", x.getNumPartitions())
print("Contenu des partitions :", x.glom().collect())
```

```
Nombre de partitions : 4
Contenu des partitions : [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15], [16, 17, 18, 19, 20]]
```

[ ]:

Capture fournie pendant l’exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 4

Partitions explicites et parallélisme par défaut

[6]:
```
x = sc.parallelize(range(1, 21), 4)

print("Nombre de partitions :", x.getNumPartitions())
print("Contenu des partitions :", x.glom().collect())
```

```
Nombre de partitions : 4
Contenu des partitions : [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15], [16, 17, 18, 19, 20]]
```

[7]:
```
x_auto = sc.parallelize(range(1, 21))

print("Partitions automatiques :", x_auto.getNumPartitions())
print("Parallélisme par défaut :", sc.defaultParallelism)
```

```
Partitions automatiques : 4
Parallélisme par défaut : 4
```

[ ]:

Capture fournie pendant l'exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 5

Fichier créé après l'erreur initiale et carrés avec map

```
[11]: with open(DATA + "/fichier_qui_n_existe_pas.txt", "w", encoding="utf-8") as f:
          f.write("Bonjour Spark\n")

[12]: inexistant.count()

[12]: 1

[13]: nombres = sc.parallelize([7, 10, 12, 17, 19], 2)

      carres = nombres.map(lambda a: a ** 2)

      print("Carrés :", carres.collect())
      print("Nombre initial :", nombres.count())
      print("Nombre après map :", carres.count())

      Carrés : [49, 100, 144, 289, 361]
      Nombre initial : 5
      Nombre après map : 5

[ ]:
```

Capture fournie pendant l'exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 6

Quotient et reste de la division par deux

```
Nombre après map : 5

[14]: couples = nombres.map(lambda a: (a // 2, a % 2))

      print(couples.collect())

[(3, 1), (5, 0), (6, 0), (8, 1), (9, 1)]

[ ]:
```

Capture fournie pendant l'exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 7

Comparaison de map, filter et flatMap

```
[14]: couples = nombres.map(lambda a: (a // 2, a % 2))

      print(couples.collect())
```

```
[(3, 1), (5, 0), (6, 0), (8, 1), (9, 1)]
```

```
[15]: pairs = nombres.filter(lambda a: a % 2 == 0)
      tests = nombres.map(lambda a: a % 2 == 0)

      print("Avec filter :", pairs.collect())
      print("Avec map :", tests.collect())
```

```
Avec filter : [10, 12]
Avec map : [False, True, True, False, False]
```

```
[16]: groupes = sc.parallelize([(1, 2), (3, 5), (8, 10)])

      plat = groupes.flatMap(lambda couple: couple)
      avec_map = groupes.map(lambda couple: couple)

      print("Avec flatMap :", plat.collect())
      print("Avec map :", avec_map.collect())
```

```
Avec flatMap : [1, 2, 3, 5, 8, 10]
Avec map : [(1, 2), (3, 5), (8, 10)]
```

```
[ ]:
```

Capture fournie pendant l’exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 8

Filtrage avec flatMap, distinct et actions

[17]:
```
pairs_flatmap = nombres.flatMap(
    lambda a: [a] if a % 2 == 0 else []
)

print(pairs_flatmap.collect())
```

```
[10, 12]
```

[18]:
```
z = sc.parallelize([1, 1, 2, 3, 4, 5, 5, 5, 6, 6])

uniques = z.distinct()

print(uniques.collect())
```

```
[4, 1, 5, 2, 6, 3]
```

[19]:
```
y = sc.parallelize([6, 8, 9, 1, 2, 3], 3)

print("count :", y.count())
print("first :", y.first())
print("take(3) :", y.take(3))
print("take(100) :", y.take(100))
print("takeOrdered(3) :", y.takeOrdered(3))
print("collect :", y.collect())
```

```
count : 6
first : 6
take(3) : [6, 8, 9]
take(100) : [6, 8, 9, 1, 2, 3]
takeOrdered(3) : [1, 2, 3]
collect : [6, 8, 9, 1, 2, 3]
```

[ ]:

Capture fournie pendant l’exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 9

Actions et agrégations numériques

```
[20]: y = sc.parallelize([6, 8, 9, 1, 2, 3], 3)

      print("count :", y.count())
      print("first :", y.first())
      print("take(3) :", y.take(3))
      print("take(100) :", y.take(100))
      print("takeOrdered(3) :", y.takeOrdered(3))
      print("collect :", y.collect())
```

```
count : 6
first : 6
take(3) : [6, 8, 9]
take(100) : [6, 8, 9, 1, 2, 3]
takeOrdered(3) : [1, 2, 3]
collect : [6, 8, 9, 1, 2, 3]
```

```
[21]: print("Somme avec reduce :", y.reduce(lambda a, b: a + b))
      print("Somme avec sum :", y.sum())
      print("Maximum :", y.max())
      print("Minimum :", y.min())
      print("Moyenne :", y.mean())
```

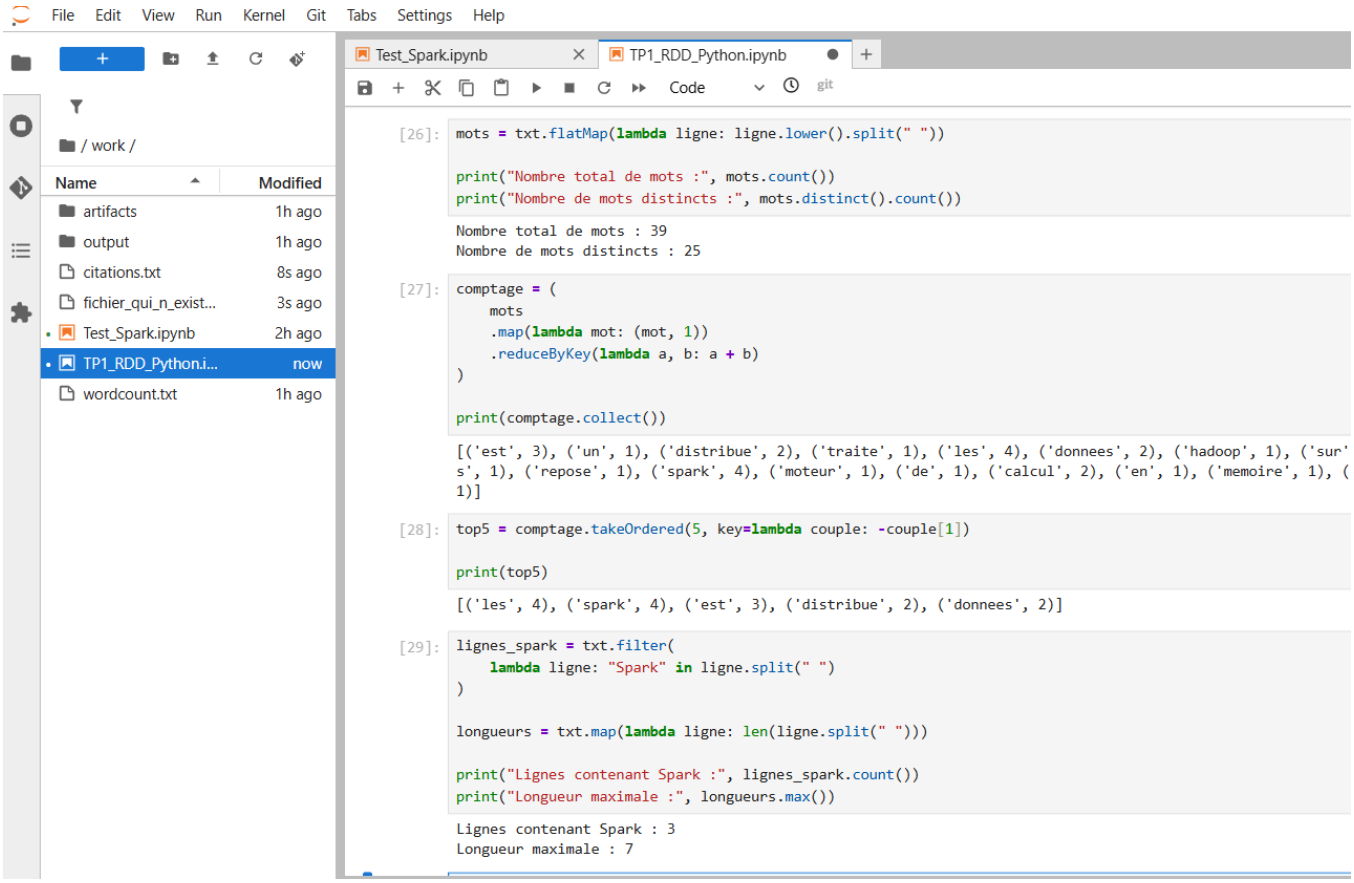
```
Somme avec reduce : 29
Somme avec sum : 29
Maximum : 9
Minimum : 1
Moyenne : 4.833333333333333
```

```
[ ]:
```

Capture fournie pendant l’exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 10

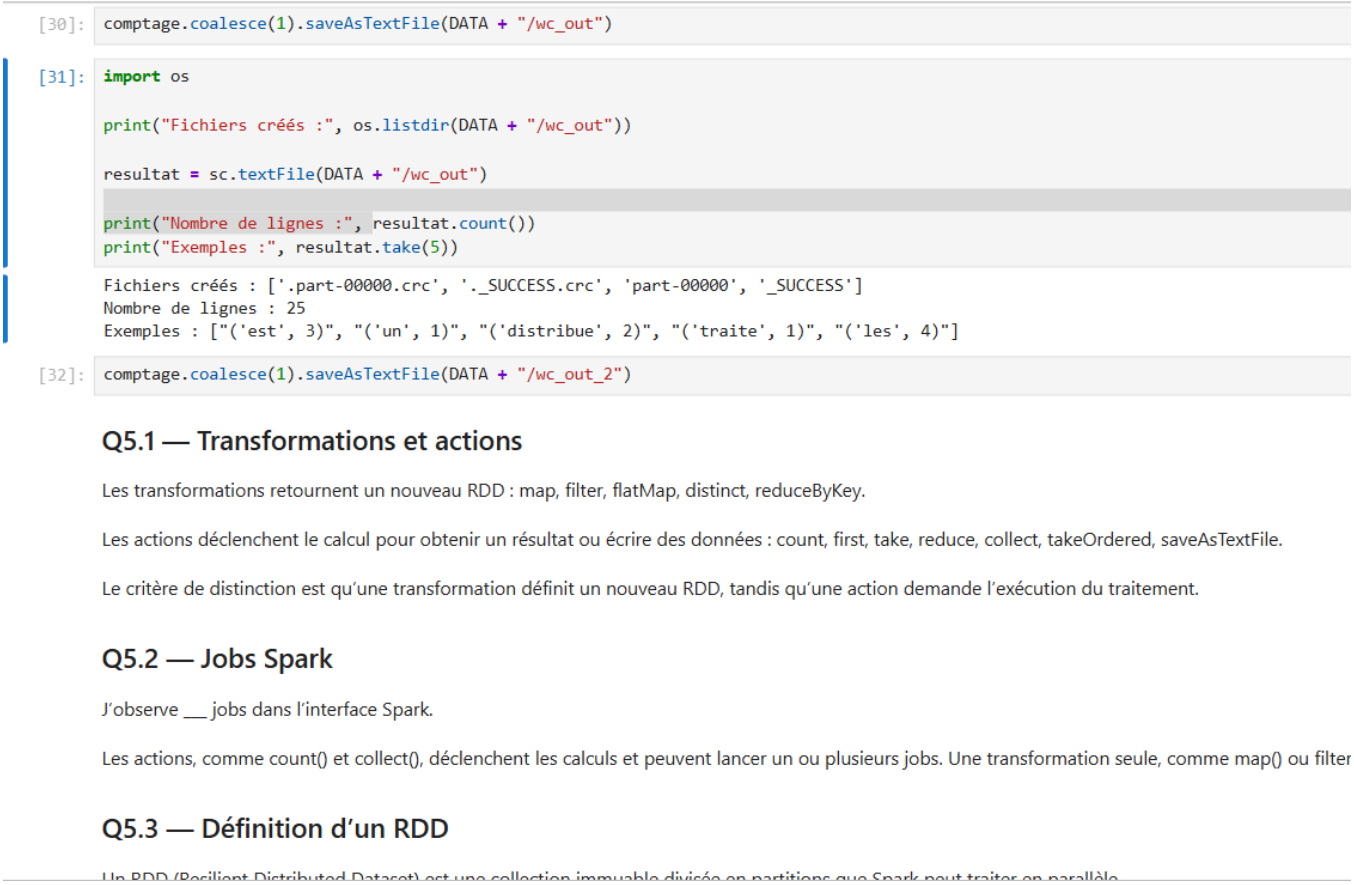
Comptage des mots, top cinq et analyse des lignes



Capture fournie pendant l'exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

Annexe - Capture 11

Sauvegarde, relecture et réponses de synthèse



```
[30]: comptage.coalesce(1).saveAsTextFile(DATA + "/wc_out")

[31]: import os

print("Fichiers créés :", os.listdir(DATA + "/wc_out"))

resultat = sc.textFile(DATA + "/wc_out")

print("Nombre de lignes :", resultat.count())
print("Exemples :", resultat.take(5))

Fichiers créés : ['.part-00000.crc', '._SUCCESS.crc', 'part-00000', '_SUCCESS']
Nombre de lignes : 25
Exemples : ["('est', 3)", "('un', 1)", "('distribue', 2)", "('traite', 1)", "('les', 4)"]

[32]: comptage.coalesce(1).saveAsTextFile(DATA + "/wc_out_2")
```

Q5.1 — Transformations et actions

Les transformations retournent un nouveau RDD : map, filter, flatMap, distinct, reduceByKey.

Les actions déclenchent le calcul pour obtenir un résultat ou écrire des données : count, first, take, reduce, collect, takeOrdered, saveAsTextFile.

Le critère de distinction est qu'une transformation définit un nouveau RDD, tandis qu'une action demande l'exécution du traitement.

Q5.2 — Jobs Spark

J'observe ___ jobs dans l'interface Spark.

Les actions, comme count() et collect(), déclenchent les calculs et peuvent lancer un ou plusieurs jobs. Une transformation seule, comme map() ou filter

Q5.3 — Définition d'un RDD

Un RDD (Resilient Distributed Dataset) est une collection immuable divisée en partitions que Spark peut traiter en parallèle

Capture fournie pendant l'exécution du TP. Les explications et les résultats sont détaillés dans les pages précédentes.

La réponse Q5.2 visible sur cette capture contient encore un emplacement vide pour le nombre de jobs ; cette valeur reste à relever dans Spark UI.